

Homework 1

Symbol-wise: we use lowercase bold like $\mathbf{x}$ to indicate a vector and uppercase bold like $\mathbf{W}$ to indicate a matrix. Others are scalars. Dimensions will be explicitly clarified if needed.

1.1 Sigmoid and Softmax Function

Sigmoid function is used in the logistic regression to predict the label $l \in \{0, 1\}$ given a sample $\mathbf{x} \in \mathbb{R}^d$. The sigmoid function is defined as follows, which maps value in $\mathbb{R}$ to a number in $(0, 1)$.

$$\sigma(s) = \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$$

Then the probabilities that label l equals to 0 or 1 are

$$P(l = 1|\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}}} = \sigma(\mathbf{w}^\top \mathbf{x}) \quad \text{and} \quad P(l = 0|\mathbf{x}) = 1 - P(l = 1|\mathbf{x}),$$

where $\mathbf{w} \in \mathbb{R}^d$ is the vector of weights. Softmax function is a generalization of logistic regression to multiple classes, i.e., $l \in \{0, 1, \dots, K-1\}$, where K denotes the number of classes. For a vector $\mathbf{z} \in \mathbb{R}^K$, softmax function normalizes it into a probability distribution by

$$\text{Softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=0}^{K-1} e^{z_j}} \text{ for } i = 0, 1, \dots, K-1 \text{ where } \mathbf{z} = (z_0, z_1, \dots, z_{K-1}) \in \mathbb{R}^K.$$

For an input vector $\mathbf{x} \in \mathbb{R}^d$, Softmax estimates the probability of each label using a weight matrix $\mathbf{W} \in \mathbb{R}^{K \times d}$ by

$$\text{Softmax}(\mathbf{W}\mathbf{x}) = \begin{bmatrix} P(l = 0|\mathbf{x}, \mathbf{w}_0) \\ P(l = 1|\mathbf{x}, \mathbf{w}_1) \\ \dots \\ P(l = K-1|\mathbf{x}, \mathbf{w}_{K-1}) \end{bmatrix} = \frac{1}{\sum_{i=0}^{K-1} e^{\mathbf{w}_i^\top \mathbf{x}}} \begin{bmatrix} e^{\mathbf{w}_0^\top \mathbf{x}} \\ e^{\mathbf{w}_1^\top \mathbf{x}} \\ \dots \\ e^{\mathbf{w}_{K-1}^\top \mathbf{x}} \end{bmatrix}.$$

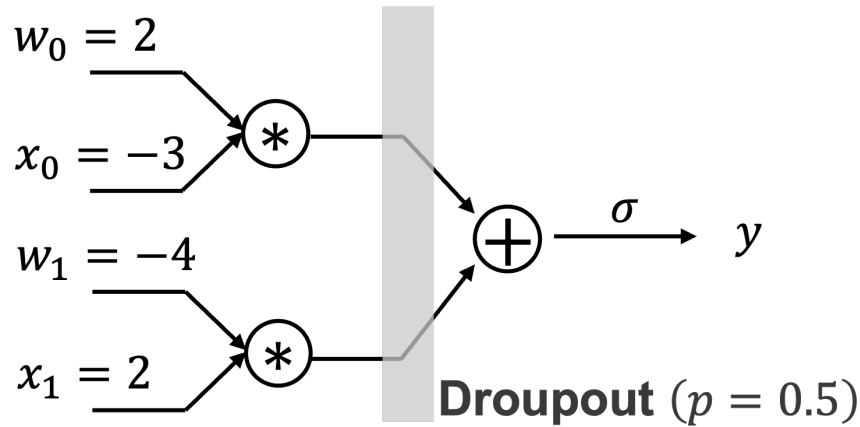
The matrix $\mathbf{W}$ is formed by the weight vectors as $\mathbf{W} = [\mathbf{w}_0, \mathbf{w}_1, \dots, \mathbf{w}_{K-1}]^\top$. It is easy to verify that the sum of all elements in the output of Softmax function is 1.

1. Prove that when $K = 2$, Softmax-based logistic regression is equivalent to Sigmoid-based logistic regression.

Hint: You may use the identity $\sigma'(s) = \sigma(s)(1 - \sigma(s))$. Softmax is invariant to adding the same scalar to all logits: $\text{Softmax}(z) = \text{Softmax}(z + c\mathbf{1})$ (proving it may be a fun exercise).

1.2 Back-Propagation

1. Let's perform back-propagation through a neural network with a sigmoid activation σ . Specifically, we insert a dropout layer before the activation. The computation graph is visualized below:



Therefore, the output $y = \sigma(\delta_0 w_0 x_0 + \delta_1 w_1 x_1)$, where $\delta_0, \delta_1 \sim \text{Bernoulli}(0.5)$. Calculate the expectation of gradients with respect to the input and parameters, i.e., $\mathbb{E}(\frac{\partial y}{\partial w_0})$, $\mathbb{E}(\frac{\partial y}{\partial x_0})$, $\mathbb{E}(\frac{\partial y}{\partial w_1})$, $\mathbb{E}(\frac{\partial y}{\partial x_1})$.

2. In a fully connected layer, let $\mathbf{y} = \mathbf{W}\mathbf{x}$, where $\mathbf{x} \in \mathbb{R}^{d_k}$ denotes the input, $\mathbf{W} \in \mathbb{R}^{d_{k+1} \times d_k}$ is the weight matrix corresponding to this fully connected layer and $\mathbf{y} \in \mathbb{R}^{d_{k+1}}$ denotes the output. $\mathcal{L}(\mathbf{y}) \in \mathbb{R}$ is the loss function. Prove that the gradient back-propagation is also in the form of a fully connected layer. In this back-propagation, the gradient of $\mathcal{L}$ with respect to $\mathbf{y}$ is the input; and the gradient of $\mathcal{L}$ with respect to $\mathbf{x}$ is the output. What is the relationship between the weight matrix of this fully connected layer and $\mathbf{W}$? (Hint: prove that $\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \tilde{\mathbf{W}} \frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ with a certain weight matrix $\tilde{\mathbf{W}}$ and figure out the relationship between $\tilde{\mathbf{W}}$ and $\mathbf{W}$. Also, please treat $\frac{\partial \mathcal{L}}{\partial \mathbf{x}}$ and $\frac{\partial \mathcal{L}}{\partial \mathbf{y}}$ as column vectors.)

3. We have a two-layer neural network as follows,

$$f(\mathbf{x}) = \sigma(\sigma(\mathbf{x} \cdot \mathbf{W}^{(1)}) \cdot \mathbf{W}^{(2)}),$$

where σ is the sigmoid function, $\mathbf{x} \in \mathbb{R}^{d_1}$ is the input, $\mathbf{W}^{(1)} \in \mathbb{R}^{d_1 \times d_2}$ and $\mathbf{W}^{(2)} \in \mathbb{R}^{d_2 \times 1}$ are weight matrices of the first and second layer, respectively. Show the gradient of the two-layer neural network's output $f(\mathbf{x})$ with respect to parameters $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(2)}$

Hint: Use the chain rule and results in Q1.2.2

1.3 Markov Chains and Google's PageRank Algorithm

Note: Markov Chains in this problem refers to discrete-time Markov chain, not continuous-time Markov chain.

A probability vector (or stochastic vector) $\vec{v}$ is a vector with non-negative entries and $\|\vec{v}\|_1 = 1$. A stochastic matrix (or Markov matrix) is a square matrix where each column is a probability vector. A distribution π is a stationary distribution of the Markov matrix P if and only of $\pi P = \pi$, *i.e.*, π is the left eigenvector of P with eigenvalue equal to 1.

In general, given an arbitrary Markov matrix, a stationary distribution may or may not exist. We can only guarantee that a stationary distribution exists if the Markov matrix is irreducible and aperiodic. In this problem, we will explore the stationary distributions of some interesting Markov matrix.

1. Consider a **weighted** graph $G = (V, E, \omega)$ where ω contains the edge weights for every edge in G . G is still irreducible and aperiodic. Show that if we were to convert G to a Markov Chain, that this Markov Chain M_G has a stationary distribution $\pi = \left(\frac{\sum_{j \in N(1)} w(1,j)}{2 \sum_{(k,j) \in E} w(k,j)}, \dots, \frac{\sum_{j \in N(n)} w(n,j)}{2 \sum_{(k,j) \in E} w(k,j)} \right)$ where $N(i)$ is the neighborhood of vertex i , and $w(i, j)$ is the weight of edge $(i, j) \in E$.
2. Again, for an arbitrary Markov matrix M , its stationary distribution may or may not exist. A stationary distribution is guaranteed to exist only if the Markov matrix is irreducible and aperiodic. PageRank algorithm calculates the “importance” of each webpage by modeling the web as a graph $G = (V, E)$, where each node $v_i \in V$ represents a webpage, and an edge $e_{ij} = (v_i, v_j)$ exists if and only if there is a link from webpage i to webpage j .

We can represent this web graph as a Markov matrix P . However, there may be pages with no outbound links, resulting in columns of P filled with zeros, which makes the Markov matrix not guaranteed to have a stationary distribution.

The PageRank algorithm modifies the matrix to ensure the existence of a stationary distribution. The steps are as follows:

- (a) Denote the original normalized adjacency matrix as P .
- (b) For any rows of zeros, replace the row by $[\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}]$, where n is the number of webpages. This yields a new matrix P' .
- (c) Construct $P'' = \alpha P' + (1 - \alpha) \frac{1}{n} ee^\top$, where $e = [1, 1, \dots, 1]^\top$, n is the number of webpages, and $\alpha \in (0, 1)$ is the damping factor.

We call P'' the **Google matrix**. Step (b) ensures that the matrix is irreducible (fully connected), and step (c) ensures that the matrix is aperiodic (introducing self-loop). The PageRank of each webpage is the stationary distribution of the Google matrix.

Calculate the stationary distribution of the web matrix (adjacency

matrix form) $A = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$ with $\alpha = 0.9$

3. Let A be a column stochastic matrix, prove “if λ is an eigenvalue of A , then $|\lambda| \leq 1$.”

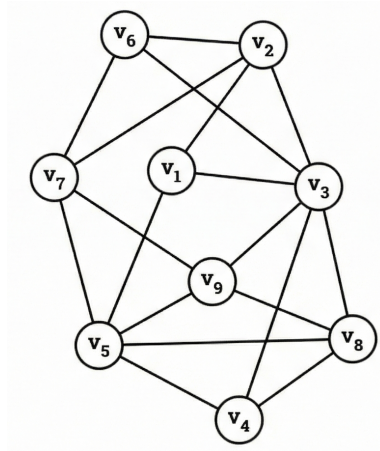
1.4 Link Prediction Scores on a Graph

Consider the following undirected, unweighted, simple graph $G = (V, E)$ with

$$V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9\}.$$

See the figure for the graph below and please refer to the definitions of Jaccard and Adamic Adar scores in the Lecture 2, slide 39. $N(u)$ is the neighborhood of node u , essentially the set of nodes directly connected to u by an edge.

1. The vertex v_1 is currently not connected to v_6, v_7, v_8 or v_9 . Compute both Jaccard and Adamic Adar for each candidate edge (v_1, u) where $u \in \{v_6, v_7, v_8, v_9\}$. Your work must explicitly list $N(v_1) \cap N(u)$ and $N(v_1) \cup N(u)$ for each v .



2. Rank the four candidate edges by (i) Jaccard score and (ii) Adamic Adar score (highest to lowest). If the rankings differ, briefly explain why that can happen.

1.5 Resources

We understand some of you may be having difficulties computing derivatives when the variables seem to be vectors or matrices. We've compiled a list of online resources you can refer to. A few of them are rather long but it'd be good to skim them to figure out when to transpose, what the dimension of the Jacobian is, etc ... essentially, introducing you to matrix calculus, which is super important in deep learning.

- <https://ccrma.stanford.edu/~dattorro/matrixcalc.pdf> (skimmable)
- <https://www.math.uwaterloo.ca/~hwolkowi/matrixcookbook.pdf> (very long)
- <https://www.cs.cmu.edu/~mgormley/courses/10601/slides/10601-matrix-calculus.pdf> (long)
- <https://atmos.washington.edu/~dennis/MatrixCalculus.pdf> (short)
- <https://arxiv.org/pdf/1802.01528> (skimmable)